



Java Code Geeks

JAVA 2 JAVA DEVELOPERS RESOURCE CENTER

Earn More on Android

50,000 Developers | \$10+ CPM's | 95%+ Fill Rates



You are here: [Home](#) [Core Java](#) [Fluent Object Creation](#)



About Luis Atencio



Fluent Object Creation

Published on January 23, 2013 | 769 view s | Filed in: Core Java Tags: Design Patterns

Java File APIs

DOC DOCX XLS XLSX PDF
MSG PPT PPTX HTML EML RTF
JPG PST Barcodes Diagrams ...
and more!



Try 30 Day Trial FREE

Many posts have been written on this subject (overwhelmingly many) but I just wanted to contribute my two-cents and write a short post about how I use the *Fluent Object Creation* pattern or object builders in Java to instantiate *Value Objects*. *Value Objects* are abstractions that are defined by their state (value) rather than their address in memory. Examples of value objects are things like money, a number, a coordinate, etc. They are used not to describe Business Objects but rather descriptions of concrete indivisible entities. Also, they make great candidates for adding them to collections and maps.

In Java, *Value Objects* should be declared final and provide no setter methods, basically making it's state immutable after creation, this is a very important

requirement. Declaring them final makes them unable to serve as parent objects. This is done by design since value objects should model small and concrete entities. The reason being is that we should be able to create and compare multiple copies of these objects, which is always done by state not by reference. In addition, you should declare proper *equals()* and *hashCode()* methods to qualify for a proper value object. In C++, the same principles apply. In C++ you should make use of the *Copy Constructor* and overload the assignment and comparison operators. The *Fluent Object Creation* pattern makes value object instantiation elegant and clean. There are many benefits that can be gained by using fluent object creation as we will see shortly.

The end result of applying this pattern from the API user's perspective will look like the following:

```
01 Money fiveEuros = new Money.Builder()
02     .currency(Currency.EURO)
03     .value(5.0L)
04     .countryOfOrigin("Spain")
05     .type("Coin")
06     .reverse("Map of Europe")
07     .obverse("Map of Spain")
08     .addColor("Bronze")
09     .addColor("Silver")
10     .year("1880")
11     .build();
```

I think you would agree that this pattern flows a lot more smoother as opposed to this:

```
01 Money fiveEuros = new Money();
02 fiveEuros.setCurrency(Currency.EURO);
03 fiveEuros.setValue(5.0L);
04 fiveEuros.countryOfOrigin("Spain");
05 fiveEuros.type("Coin");
06 fiveEuros.reverse("Map of Europe");
07 fiveEuros.obverse("Map of Spain");
08
09 List<String> colors = new ArrayList<String>();
10 for(String color: new String[] { "Bronze", "Silver" }) {
11     colors.add(color);
12 }
13
14 fiveEuros.setColors(colors);
15 fiveEuros.setYear("1880");
```

Which seems broken and has lots of typing and repetition. This is an example of building a pretty big value object in my opinion, most of tend to be very small. Before we talk about the benefits of creating objects , let's have a look at the structure of this pattern:

```
01 public final class Money {
02
03     private Long value;
04     private String countryOfOrigin;
```



Monitor Cloud & App Performance



Find your startup



Solid Database Apps



whitetec

Why Static Analysis Fai
[alm.parasoft.com/Static-Analysi](#)
Learn the top 10 mistakes to Read this before starting SA

Kaspersky™ Brasil
[www.Kaspersky.com](#)
Loja Oficial de Antivírus. Baixe A Edição 2013 Hoje!

Stellaris Code Sample
[TI.com/Stellarisware](#)
Free License & Royalty-Free Use Easy, Full-Function Code Creati

Join Us

227,650 Dec unique

Do you have a blog with unique content? Then, check out our JC program. You can also be a gue Code Geeks

Tags

Akka Android Tutorial Apache Akka
Apache Maven Apache T
Big Data Books Cache Cloud
Design Patterns Eclipse
Google Guava Google GWT
JavaFX JavaOne J
JBoss Hibernate JPA
JVM Logging MapReduce
Performance Performanc
Play Framework Project
RESTful Web Service
Spring Spring MV
Testing x

```

05     private Currency currency;
06     private String type;
07     private String reverse;
08     private String obverse;
09     private List<String> colors;
10     private Date year;
11
12     private Money() { }
13
14     // -- getters, hashCode, equals -- //
15
16     // Static inner Builder class
17     public static class Builder {
18         private Money _temp = new Money();
19     public Builder value(Long val) {
20         _temp.value = val;
21     return this;
22     }
23     public Builder countryOfOrigin(String countryOfOrigin) {
24         _temp.contryOfOrigin = countryOfOrigin;
25         return this;
26     }
27     public Builder currency(Currency c) {
28         _temp.currency = c;
29     return this;
30     }
31     public Builder type(String t) {
32         _temp.type = t;
33     return this;
34     }
35     public Builder reverse(String r) {
36         _temp.reverse = r;
37     return this;
38     }
39
40
41     public Builder obverse(String o) {
42         _temp.obverse = o;
43     return this;
44     }
45
46
47     public Builder addColor(String c) {
48     if(_temp.colors == null) {
49         _temp.colors = new ArrayList<String>();
50     }
51     _temp.colors.add(c);
52     return this;
53 }
54
55     public Builder year(String y) {
56         if(y == null || y.isEmpty()) {
57             _temp.year = new Date();
58         }
59         else {
60             _temp.year = DateFormat.parse(y);
61         }
62     return this;
63 }
64     public Money build() {
65         // Validate object
66         if(Strings.isNullOrEmpty(_temp.name) || _temp.currency == null) {
67             throw new IllegalArgumentException("Coin currency and value required");
68         }
69     return _temp;
70 }
71 }
72 }

```

This is also a matter of taste, but I prefer the *static inner class* approach. I like the canonical nature of referring to the builder as *Money.Builder*. Also making it *static* is required since the builder instance needs to live independently of the enclosing class. I like this pattern because it has the following benefits:

1. **Greater object encapsulation:** I can easily enforce object construction using builders by making the *Money* constructor private (this is just stylistic). This completely hides all of the intricacies of creating this object: list creation, date parsing, etc. From the user's perspective, what we end up with is an object that is simple to instantiate. My illustration is a very simple one, but imagine more complex object graphs.
2. **Code readability:** Using this pattern to create objects, makes unit tests and code very easy to read and follow.
3. **Less typing in the long run:** Even though you have to add an extra builder method for every added attributes, the amount of typing you save in the long run makes it worth while.

Conclusion

Using the fluent creation pattern is more work up front, but the benefits of having it pays off at the end. It makes instantiating objects very elegant and clean. You don't have to use it with *Value Objects*, most of the benefit of using *Fluent Object Creation* is when you need to build pretty complex object graphs, but I wanted to show that it can also suit small value objects.

Reference: [Fluent Object Creation](#) from our [JCG partner](#) Luis Atencio at the [Reflective Thought](#) blog.

7121

ALSO ON JAVA CODE GEEKS

[Inferred exceptions in Java](#) 5 comments

[15 Online Learning websites that you should check out](#) 5 comments

RECOMMENDED CONTENT

What's this?

[Selling Banana Slicers \(and More!\) With Gag Reviews](#) Businessweek

[The Most Beautiful Garden in New England](#) Gardenista

2 comments • 2 reactions

★ 0



Leave a message...

Discussion | Community

Share



Sean Loughran • a day ago

Maybe because I never used it, but I'm not a huge fan of the Builder Pattern. I also wouldn't use all those setters in the first, if you need N things to describe an object, why not pass them in the Constructor

Money fiveEuros = new Money(Currency.Euro,5.0L, "Spain", "Coin", "Map of Europe", "Map of Spain",new String[] {"Bronze", "Silver"}, 1180);

I don't think anything is wrong with the Builder per se, but I like to be able to look at the constructor and quickly see all the parameters needed, as well as their types.

Good article though, I like seeing the innards of one of the Builder objects.

0 ^ | v • Reply • Share



Marco Castigliego • a day ago

Nice article, I like the builder pattern, but I think it lacks of some creational logic, for example, a client can do Money.Builder().build(); which will return an inconsistent Object. I guess the step builder pattern (<http://rdafbn.blogspot.it/2012...> gives a better control of the object creation, and as you said "is more work up front, but the benefits of having it pays off at the end".

0 ^ | v • Reply • Share

 Comment feed

 Subscribe via email

Knowledge Base	Partners	Hall Of Fame	About Java Code Geeks	
Examples	Mkyong	“Android Full Application Tutorial” series	JCGs (Java Code Geeks) is an independent online community creating the ultimate Java to Java developers resource center: technical architect, technical team lead (senior developer), principal developers alike. JCGs serve the Java, SOA, Agile and communities with daily news written by domain experts, article announcements, code snippets and open source projects. License  This work by Java Code Geeks is licensed under a Creative Commons Attribution-ShareAlike 3.0 Unported License.	
Resources	The Code Geeks Network	GWT 2 Spring 3 JPA 2 Hibernate 3.5 Tutorial		
Software		Android Game Development Tutorials		
Tutorials		Android Google Maps Tutorial		
		Android Location Based Services Application – GPS location		
	Java Code Geeks	Funny Source Code Comments		
	.NET Code Geeks	Java Best Practices – Vector vs ArrayList vs HashSet		
	Web Code Geeks	Android JSON Parsing with Gson Tutorial		
		Android Quick Preferences Tutorial		